

ADR-001：采用前后端分离 + 模块化单体架构

状态

已接受

背景

团队为学生开发人员，技术能力偏向基础全栈开发。需要兼顾开发效率、模块分工、部署简易度，同时满足任务发布、抢单、订单流转、消息通知等核心业务，暂不具备大规模微服务拆分与运维的条件。

决策

采用前后端分离 + 模块化单体架构，前端独立开发部署，后端按业务域拆分为多模块的单体应用，通过 RESTful API 完成前后端交互。

理由

前后端分离可让前端专注页面交互，后端专注业务逻辑，分工明确适配学生团队协作模式。

模块化单体在开发期可快速迭代、统一调试，避免微服务带来的服务注册、配置中心、链路追踪等复杂运维成本。

10 周开发周期内，模块化单体可快速完成业务落地，后续业务扩容时可逐步拆分为微服务，具备良好演进性。

后端按用户、任务、订单、消息等域拆分模块，符合业务边界，便于多人并行开发。

后果

正面影响：开发部署简单、调试成本低、团队上手快，能按期完成课程项目交付。

负面影响：单体应用在极端高并发场景下性能瓶颈明显，无法独立扩容单个业务模块。

需要关注的风险：模块间若不合理依赖，后期会出现代码耦合，难以维护。

AI 辅助记录

AI 初稿内容摘要：AI 建议直接采用微服务架构，强调高可用与可扩展性，未考虑 10

周开发周期与学生团队能力约束。

人工修订内容：调整为前后端分离 + 模块化单体，补充开发周期、团队能力、部署成本等约束条件。

修订理由：微服务架构超出当前团队能力与项目周期，模块化单体更符合校园课程项目的实际场景。

ADR-002：选用 Vue3 + Element Plus 作为前端技术栈

状态

已接受

背景

前端需要实现响应式页面、组件化复用、状态管理、路由控制，适配校园用户桌面端浏览，团队具备基础 Vue 使用经验，需选择学习成本低、组件生态完善的前端框架与 UI 库。

决策

前端采用 Vue3 作为核心框架，搭配 Element Plus UI 组件库、Pinia 状态管理、Axios 网络请求，基于 Vite 构建。

理由

Vue3 组合式 API 易于理解，学生团队学习成本低，能快速完成页面开发。

Element Plus 提供完善的表单、弹窗、导航、卡片等组件，直接适配任务发布、订单管理、个人中心等业务页面，减少重复开发。

Pinia 轻量易用，适配 Vue3，可高效管理登录状态、全局消息、信息流数据。

Vite 构建速度快，开发环境热更新高效，提升开发体验。

后果

正面影响：组件复用率高、开发速度快、页面交互体验统一，满足项目前端所有业务

需求。

负面影响：Element Plus 组件样式定制成本较高，复杂交互需额外自定义开发。

需要关注的风险：Vue3 与部分旧插件不兼容，需控制依赖版本。

AI 辅助记录

AI 初稿内容摘要：AI 推荐 React+Ant Design，强调生态成熟。

人工修订内容：替换为 Vue3+Element Plus，补充团队技术背景、项目开发效率、组件适配性等理由。

修订理由：团队更熟悉 Vue，Vue3+Element Plus 更适合快速开发校园类 Web 项目。

ADR-003：选用 Redis 作为缓存中间件处理高并发与会话存储

状态

已接受

背景

校园互助平台存在抢单高并发场景，同时需要存储 JWT 令牌、会话状态、热点任务数据，要求高性能、轻量的缓存组件，且部署与配置简单，适配学生开发与部署环境。

决策

采用 Redis 作为缓存中间件，结合 Redisson 实现分布式锁，用于抢单并发控制、热点数据缓存、JWT 会话存储。

理由

Redis 读写性能极高，可支撑任务抢单、列表查询等高并发场景，减轻 PostgreSQL 数据库压力。

Redisson 分布式锁可保证抢单业务的原子性，避免超卖、重复接单问题。

Redis 支持键值过期策略，适合存储临时会话与限时任务数据。

后果

正面影响：有效解决抢单并发安全问题，提升接口响应速度，降低数据库查询负载。

负面影响：Redis 数据持久化配置不当可能导致缓存数据丢失，需合理设置策略。

需要关注的风险：缓存穿透、缓存雪崩问题，需在开发中增加防护逻辑。

AI 辅助记录

AI 初稿内容摘要：AI 仅说明 Redis 用于缓存，未提及并发抢单、分布式锁、会话存储等项目具体场景。

人工修订内容：补充抢单并发控制、Redisson 分布式锁、JWT 会话存储等业务相关用途，完善风险与后果。

修订理由：贴合校园互助平台抢单核心业务，明确 Redis 在项目中的实际作用，而非通用缓存描述。

ADR-004：选用 PostgreSQL 作为主关系型数据库

状态

已接受

背景

CampusHub 校园互助平台需要存储用户信息、任务数据、订单记录、评价内容、论坛帖子等结构化数据，要求支持事务 ACID 特性、复杂查询、JSON 数据类型（用于存储图片列表、扩展字段），同时需开源免费、部署简单，适配学生团队的运维能力。

决策

我们决定采用 PostgreSQL 作为主关系型数据库，用于存储所有核心业务结构化数据。

理由

PostgreSQL 完全开源免费，无商用授权成本，适合学生课程项目。

支持完整的事务 ACID 特性，保证订单流转、用户状态变更等核心数据的一致性。

原生支持 JSON/JSONB 数据类型，可灵活存储任务图片列表、论坛媒体资源等半结构化数据，无需额外设计关联表。

支持复杂查询、索引优化，能满足任务列表筛选、关键词搜索、个人主页聚合查询等业务需求。

内置 PostGIS，可用于“附近的人”功能，便于后续扩展地理定位方面功能。

后果

正面影响：数据存储稳定可靠，灵活支持结构化与半结构化数据，满足项目所有业务数据存储需求。

负面影响：相比 MySQL，PostgreSQL 在国内学生团队中的普及度略低，部分成员需短暂学习适应。

需要关注的风险：需合理设计索引，避免后期数据量增长导致查询性能下降。

AI 辅助记录

AI 初稿内容摘要：AI 推荐 MySQL，强调普及度高，未重点提及 PostgreSQL 的 JSON 支持与复杂查询优势。

人工修订内容：替换为 PostgreSQL，补充 JSON 数据类型、复杂查询、事务一致性、地理查询等项目具体需求理由。

修订理由：项目中任务图片、论坛媒体等半结构化数据存储需求明确，PostgreSQL 的 JSONB 类型比 Mysql 的 JSON 支持更高效，复杂查询能力更适配业务场景。